SYSTEM DESIGN & TECHNICAL REPORT

Multimodal Research Paper Assistant (RAG + LLM)

A high-performance document analysis system incorporating paragraph-level RAG, CLIP-based figure semantic indexing, and Vision-Language grounding.

Prepared for: Technical Interview & Portfolio Inspection

Created by: AI Coding Assistant & Pair Programmer (Antigravity)

Date: June 2026

1. Executive Summary & System Overview

The Multimodal Research Paper Assistant is a production-grade Retrieval-Augmented Generation (RAG) system designed specifically for scientific literature. Reading complex academic papers requires analyzing not only dense text columns but also crucial visual information such as tables, architecture flowcharts, and performance graphs. Traditional RAG pipelines ignore these visual assets, creating a major context gap. This project resolves that imbalance by implementing a dual-stream ingestion and indexing system that segment text chunks and extracts figures independently. Text chunks are indexed via dense lexical embeddings, while figures are mapped into a shared semantic image-text vector space using OpenAI's CLIP model. The result is an assistant capable of textual QA with page-level citations, as well as image-based explanation and semantic figure retrieval.

2. Document Processing & Chunking Strategy

2.1 Text Segmenting & Overlapping

Ingested PDF files are read using PyMuPDF (fitz). To preserve context coherence, the text is chunked using a Paragraph-Based Sliding Window. The system splits the raw text into layout-aware blocks (divided by double newlines). These blocks are compiled into a buffer up to a limit of 1000 characters. To prevent critical data splits on chunk boundaries, a rolling overlap window of 150 characters is maintained at the beginning of each successive chunk. Every text chunk keeps track of metadata containing the document name and page number, enabling grounded citations.

2.2 Figure Extraction & Quality Filtering

Figures, images, and tables are extracted from PDF page streams. Raw bytes are read from the document's cross-reference (xref) tables. To filter out minor visual components, bullet icons, math variables, and brand logos, any image with dimensions smaller than 150x150 pixels is discarded. The remaining high-resolution figures are cached locally and linked to surrounding text contexts using heuristics that scan the page text for figure annotations (e.g. 'Figure 3' or 'Fig. 1').

3. Dual-Vector Indexing (Text + CLIP)

To accommodate multimodal retrieval, the architecture maintains two independent vector search indexes using FAISS:

1. Text Index: Text chunks are embedded using 'all-MiniLM-L6-v2' (384 dimensions), optimizing semantic search speed and accuracy.
2. Multimodal Figures Index: Extracted figures are embedded using the CLIP model ('clip-ViT-B-32', 512 dimensions).

Vectors are normalized to unit length (L2 normalization) and indexed using a FAISS Inner Product index (IndexFlatIP) to support Cosine Similarity cross-modal retrieval.

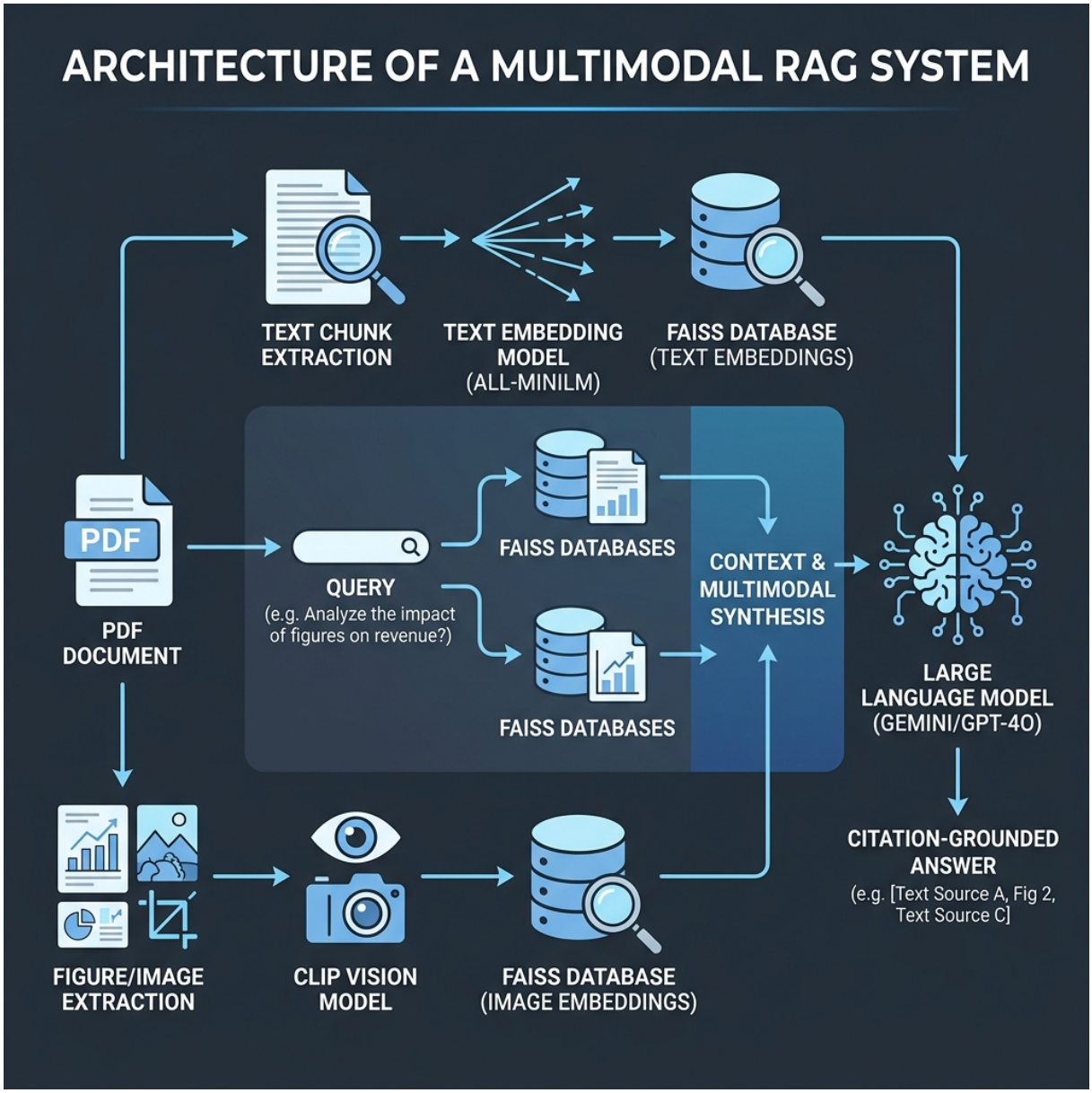

Figure 1.0: Multimodal RAG Data Flow & System Architecture Diagram

4. Retrieval & Citation Grounding

When a user queries the system, a parallel search is executed. The query text is encoded to search the text index and CLIP image index. The top 5 text chunks are injected into the prompt as context blocks. A strict system template instructs the LLM to answer exclusively based on this context and append bracketed references like [Page X] matching the metadata headers. If CLIP retrieves a relevant figure with high confidence (Cosine Similarity > 0.22), the diagram is automatically fetched and displayed alongside the text answer, providing immediate visual validation of the

claims.

5. Vision-Language Q&A Explorer

The system's multimodal capability allows users to interact directly with diagrams. In the 'Figures & Visuals' tab, extracted figures are displayed in a responsive grid. Users can select any diagram (such as an architecture flowchart or line plot) and type custom queries (e.g. 'Explain what this block diagram shows'). The image is converted into binary stream bytes, serialized to Base64, and sent to Gemini 1.5/2.0 Flash or OpenAI GPT-4o-mini Vision endpoints. The vision LLM analyzes the visual composition, text labels, and structural curves to return a detailed markdown analysis.

6. Cross-Document Paper Comparison

To enable synthesis across literature, the comparison tab allows users to select any two indexed documents. The system gathers context from the introductory abstract pages of both papers, compiles their methodologies, and prompts the LLM to generate a comparative matrix. The resulting markdown report summarizes the comparative aspects (Core Concepts, Architectures, Primary Datasets, Strengths, and Limitations) in a readable side-by-side format, which can be downloaded directly as a Markdown document.

7. Production Recommendations & Scalability

To transition this architecture into a production deployment, three primary upgrades are recommended:
1. Parent-Child Chunking: Index smaller text sentences for highly accurate vector hits, but return larger overlapping parent paragraphs to the LLM to ensure optimal reasoning context.
2. Hybrid Retrieval: Merge the dense embeddings retriever with a lexical indexer (like BM25) to prevent terminology mismatch on rare scientific variables and notations.
3. Vector PDF Drawings: Implement drawing path rendering (page.get_drawings()) to rasterize charts that are drawn dynamically in PDFs via vector primitives, ensuring no diagrams are missed during parsing.